DRDO Offline Multimodal Intelligence Analysis System

Project Catch-Up Document (Current Status)

1. Project Vision

The goal of this project is not to build a simple chatbot over documents. Instead, we are designing an offline intelligence-analysis platform capable of processing security-related information and helping analysts discover patterns, similarities, and trends across incidents.

The system must operate completely offline without using external APIs such as OpenAI, Gemini, Claude, or any cloud-hosted AI services.

The intended use case is intelligence analysis involving:

- Terrorist attacks
- Infrastructure attacks
- Explosive incidents
- Security events
- Public safety incidents
- Potential future intelligence datasets

The system should eventually answer questions such as:

- Find incidents similar to the 2019 Sri Lanka Easter Attacks.
- Show attacks involving explosives.
- Compare attacks targeting transportation infrastructure.
- Identify trends in coordinated attacks.

The important realization during our design discussions was that analysts think in terms of incidents, not document chunks.

Traditional RAG systems retrieve chunks of text.

Our system will retrieve incidents.

This distinction influences the entire architecture.

---

2. High-Level System Architecture

Current architecture:

Data Sources

↓

Document Ingestion

↓

Text Extraction / OCR

↓

Document Normalization

↓

Unified Document Object

↓

Incident Extraction

↓

Incident Records

↓

Embeddings + Metadata

↓

FAISS

↓

Hybrid Retrieval

↓

Analysis Layer

↓

Local LLM Reports

At present we are only focusing on the first part of this pipeline:

Document Ingestion and Standardization.

---

3. Why Data Ingestion Is Important

Most beginner RAG projects immediately jump into:

PDF → Chunk → Embedding

However, this approach becomes problematic when handling multiple document formats.

Our project must support:

- PDF files
- DOCX files
- News articles
- Images
- Scanned PDFs
- Future multimedia sources

If each file type is handled differently throughout the pipeline, the codebase becomes difficult to maintain.

Therefore, we need a common representation for every document entering the system.

Before incident extraction, every input must be converted into a unified format.

This is the primary objective of Phase 1.

---

4. Understanding Different Document Types

One major realization was that not all PDFs are the same.

We identified three major categories.

Type 1: Digital PDF

Examples:

- Reuters article PDF
- Exported reports
- Computer-generated PDFs

Characteristics:

- Text can be selected
- Text already exists inside the PDF

Extraction Method:

- PyMuPDF
- pdfplumber

No OCR is required.

Type 2: Scanned PDF

Examples:

- Scanned intelligence reports
- Old archived documents
- Printed documents converted to PDF

Characteristics:

- Pages are essentially images
- Text cannot be selected

Extraction Method:

- OCR

Possible tools:

- EasyOCR
- Tesseract
- OCRmyPDF

Type 3: Mixed PDF

Examples:

- Reports containing both text pages and scanned annexures

Characteristics:

- Some pages contain digital text
- Some pages require OCR

This is likely to be common in real-world intelligence datasets.

---

5. Phase 1A – File Type Detection

Before extracting any text, the system must determine what kind of file has been received.

Input files may include:

- .pdf
- .docx
- .txt
- .jpg
- .jpeg
- .png

The first component of the ingestion pipeline will therefore be a file classifier.

Example:

attack_report.pdf

↓

File Detection Module

↓

PDF Loader

Similarly:

incident_scan.jpg

↓

File Detection Module

↓

OCR Pipeline

The purpose is to route every file to the correct extraction method.

---

6. Text Extraction Layer

Once the document type is known, text extraction begins.

Examples:

Digital PDF

↓

PyMuPDF

↓

Extracted Text

DOCX

↓

python-docx

↓

Extracted Text

Image

↓

EasyOCR

↓

Extracted Text

At this stage the system only cares about obtaining readable text.

No intelligence extraction happens yet.

---

7. Why We Need A Unified Document Object

This was one of the most important design decisions.

Suppose we process:

- PDF
- DOCX
- Image

If later components need to know where the document came from, the system becomes complicated.

Instead, every loader should output the same internal structure.

This structure is called:

Document

The Document object becomes the universal language spoken inside the system.

Everything downstream only interacts with Document objects.

---

8. Designing Our Own Document Format

We decided to create a custom Python dataclass.

The purpose is to convert every source format into a common representation.

Conceptually:

PDF

↓

Document

DOCX

↓

Document

Image

↓

Document

The extraction pipeline no longer cares about the original source.

It only cares about the resulting Document object.

A preliminary design is:

```
from dataclasses import dataclass, field

@dataclass
class Document:

    doc_id: str

    source_type: str

    source_path: str
```

```
    raw_text: str

    title: str = ""

    language: str = ""

    page_count: int = 0

    metadata: dict = field(default_factory=dict)
```

Example:

```
Document(
    doc_id="001",
    source_type="pdf",
    source_path="srilanka_attack.pdf",
    raw_text="Attack occurred in Colombo..."
)
```

The key idea is that every loader returns the same object.

---

9. Why Dataclasses Were Chosen

A dataclass provides a clean and structured way of representing information.

Advantages:

- Easy to read
- Easy to maintain
- Type-safe
- Better than large dictionaries
- Easier debugging
- Suitable for future expansion

As the project grows, additional fields can be added without changing the overall architecture.

---

10. Document vs Incident

Another major architectural decision was separating documents from incidents.

Document:

Represents source material.

Examples:

- Reuters article
- Government report
- PDF file

Incident:

Represents the real-world event described by the document.

Examples:

- Sri Lanka Easter Attack
- Mumbai Attack
- Railway bombing

The future workflow will be:

Source File

↓

Document Object

↓

Incident Extraction

↓

Incident Object

↓

Embeddings

↓

FAISS

This separation prevents confusion and mirrors real intelligence systems.

---

11. Future Phase After Data Ingestion

Once Document objects are reliably produced, Phase 2 begins.

Phase 2 focuses on extracting structured intelligence from documents.

Example Incident Record:

```
{
  "incident_id": "",
  "date": "",
  "country": "",
  "city": "",
  "attack_types": [],
  "weapon_types": [],
  "target_types": [],
  "killed": 0,
  "injured": 0,
  "responsible_groups": [],
  "summary": "",
  "retrieval_text": ""
}
```

This incident record will eventually become the unit of retrieval instead of document chunks.

---

12. Current Conclusion

At the current stage of the project, no embeddings, vector databases, FAISS indexes, or LLM reasoning should be implemented yet.

The immediate goal is to build a reliable ingestion system capable of:

1. Accepting multiple document formats.
2. Identifying document types.
3. Extracting text correctly.
4. Performing OCR when needed.
5. Producing a unified Document object.
6. Providing clean input for future incident extraction.

The success of every later stage depends on the quality of this ingestion pipeline. Therefore, Phase 1 should focus entirely on building a robust and scalable document-processing foundation before moving to incident extraction or retrieval.